

Falaa cidade!

Cadastro, login com e-mail e senha,
verificação de e-mail, recuperação de senha e logout

Branch: **aula19102026** (criada a partir da **main**, commit 9ad608e)
Data: 19/10/2026

1. Resumo

Antes, o app só tinha login com o Google. Nesta aula ele ganhou:

- **Cadastro** com nome, e-mail e senha.
- **Verificação de e-mail:** o app só libera o acesso depois que o usuário clica no link enviado para o e-mail dele. Assim, um e-mail inventado nunca entra.
- **Login com e-mail e senha** na tela inicial, ao lado do login com o Google.
- **Recuperação de senha** ("Esqueci minha senha").
- **Botão Sair** no Perfil, que volta para a tela inicial e apaga o histórico de navegação.
- **Ajustes visuais:** o botão "Entrar" ficou azul e o botão do Google ganhou as cores oficiais.

Tudo usa o **Firebase Authentication**, que o projeto já tinha. Nenhuma biblioteca nova foi instalada.

2. Arquivos alterados

Arquivo	Situação	O que mudou
`app/cadastro.tsx`	Novo	Formulário de cadastro. Cria a conta e envia o e-mail de confirmação.
`app/verificar-email.tsx`	Novo	Tela de espera até o e-mail ser confirmado.
`app/esqueci-senha.tsx`	Novo	Envia o link para criar uma nova senha.
`constants/erros-firebase.ts`	Novo	Traduz os erros do Firebase para português.
`docs/cadastro-login-email.md`	Novo	Passo a passo desta aula, em Markdown.
`app/index.tsx`	Alterado	Campos de e-mail e senha, botão Entrar azul, links "Esqueci minha senha" e "Cadastre-se", botão do Google com o visual oficial.
`app/_layout.tsx`	Alterado	Removeu as rotas antigas (modal, mapa, perfil) e registrou as três telas novas.
`app/(tabs)/perfil.tsx`	Alterado	Botão Sair.
`context/auth-context.tsx`	Alterado	`signOut()` agora devolve uma Promise.
`hooks/use-auth.ts`	Alterado	Novo hook `useSair()`.
`styles/index.tsx`	Alterado	Novas cores: `azul`, `googleBorda` e `googleTexto`.

3. Antes de tudo: ativar o e-mail/senha no Firebase

Isso é feito **uma vez só**, no console do Firebase:

- Abra console.firebase.google.com e escolha o projeto.
- Vá em **Authentication** → **Sign-in method**.

- Clique em **E-mail/senha**, ative a primeira opção e salve. Deixe o "link de e-mail (sem senha)" desligado.
- (Opcional) Em **Authentication** → **Templates**, ajuste o texto dos e-mails de verificação e de redefinição de senha.

Sem esse passo, o cadastro dá o erro `auth/operation-not-allowed`.

4. O fluxo completo

```
                  "Cadastre-se"
  [ index ] ──────────────────────────> [ cadastro ]
     │                                      │  cria a conta e envia o e-mail
     │  "Entrar" (e-mail e senha)           ▼
     ├── e-mail NÃO confirmado ──> [ verificar-email ] ── "Já confirmei" ──> [ mapa ]
     ├── e-mail confirmado ──────────────────────────────────────────────────> [ mapa ]
     │
     └── "Esqueci minha senha" ──> [ esqueci-senha ]  (link chega por e-mail)

  [ perfil ] ── "Sair" ──> [ index ]   (histórico apagado)
```

5. O que foi feito, passo a passo

5.1 Branch e limpeza do Stack raiz

Criamos a branch `aula19102026` a partir da `main`. O primeiro commit (`55afddd`) tirou do `app/_layout.tsx` as rotas `modal`, `mapa` e `perfil`. Elas já tinham ido para dentro de `(tabs)` na versão anterior. Depois registramos as telas novas:

app/_layout.tsx

```
<Stack>
  <Stack.Screen name="index" options={{ headerShown: false }} />
  <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
  <Stack.Screen name="cadastro" options={{ title: "Cadastro" }} />
  <Stack.Screen name="esqueci-senha" options={{ title: "Recuperar senha" }} />
  <Stack.Screen name="verificar-email" options={{ headerShown: false }} />
</Stack>
```

5.2 Cadastro

A tela `app/cadastro.tsx` tem os campos nome, e-mail, senha e confirmar senha. Antes de chamar o Firebase, ela confere se tudo foi preenchido e se as duas senhas são iguais. Depois:

app/cadastro.tsx

```
const { user } = await createUserWithEmailAndPassword(auth, email.trim(), senha);
await updateProfile(user, { displayName: nome.trim() });  // salva o nome

auth.languageCode = "pt-BR";        // e-mail de confirmação em português
await sendEmailVerification(user);  // manda o link

router.replace("/verificar-email"); // "voltar" não reabre o formulário
```

Depois do `createUserWithEmailAndPassword`, o usuário **já está logado**, mas com `user.emailVerified === false`.

5.3 Verificação de e-mail

A tela `app/verificar-email.tsx` mostra para qual e-mail o link foi enviado. Ela tem três botões: **Já confirmei**, **Reenviar e-mail** e **Usar outra conta**.

O Firebase **não avisa o app** quando o link é clicado. Por isso, o botão "Já confirmei" pede os dados atualizados ao servidor:

app/verificar-email.tsx

```
await auth.currentUser.reload();   // busca o usuário atualizado no servidor

if (auth.currentUser.emailVerified) {
  router.replace("/mapa");
} else {
  setMensagem("Ainda não confirmado. Abra o link que enviamos no seu e-mail.");
}
```

5.4 Login com e-mail e senha

A tela inicial (`app/index.tsx`) ganhou os campos de e-mail e senha e o botão **Entrar**. O login em si é uma linha:

```
await signInWithEmailAndPassword(auth, email.trim(), senha);
```

Quem decide para onde ir é o `useEffect` que observa o `user`. Ele vale para os dois tipos de login. As contas Google já chegam com `emailVerified = true`:

app/index.tsx

```
useEffect(() => {
  if (user) {
    router.replace(user.emailVerified ? "/mapa" : "/verificar-email");
  }
}, [user]);
```

5.5 Recuperação de senha

O link **Esqueci minha senha** fica abaixo do campo de senha. Ele abre `app/esqueci-senha.tsx` e leva junto o e-mail que já estava digitado:

```
<Link href={{ pathname: "/esqueci-senha", params: { email: email.trim() } }} asChild>

// na tela esqueci-senha:
const params = useLocalSearchParams<{ email?: string }>();
await sendPasswordResetEmail(auth, email.trim());
```

A nova senha é escolhida numa **página do próprio Firebase**, então o app não precisa de uma tela de "nova senha". A mensagem de sucesso é a mesma **exista ou não a conta**. Assim, ninguém consegue usar a tela para descobrir quais e-mails estão cadastrados.

5.6 Mensagens de erro em português

O Firebase devolve códigos como `auth/email-already-in-use`. O arquivo `constants/erros-firebase.ts` troca esses códigos por textos amigáveis, e todas as telas usam o mesmo arquivo:

Código do Firebase	Mensagem mostrada
`auth/email-already-in-use`	Este e-mail já está cadastrado.
`auth/invalid-email`	E-mail inválido.
`auth/weak-password`	A senha precisa ter pelo menos 6 caracteres.
`auth/invalid-credential`	E-mail ou senha incorretos.
`auth/too-many-requests`	Muitas tentativas. Aguarde um pouco e tente de novo.
`auth/network-request-failed`	Sem conexão com a internet.
(qualquer outro)	Algo deu errado. Tente novamente.

5.7 Botão Sair (logout)

O Perfil ganhou o botão **Sair**. A primeira versão tinha um bug: o app ia para o `index` antes de o Firebase terminar o logout. Como o `user` ainda existia, o `useEffect` do `index` mandava o usuário **de volta para o mapa**. A correção teve duas partes:

- `signOut()`, no `context/auth-context.tsx`, passou a devolver a Promise do Firebase, para quem chama poder esperar.
- Criamos o hook `useSair()` em `hooks/use-auth.ts`. Ele espera o logout, fecha todas as telas abertas e troca a que sobrar pelo `index`.

hooks/use-auth.ts

```
export function useSair() {
  const { signOut } = useAuth();

  return async function sair() {
    await signOut();               // 1. espera o Firebase (user vira null)
    if (router.canDismiss()) {
      router.dismissAll();         // 2. fecha as telas empilhadas
    }
    router.replace("/");           // 3. só o index fica na pilha
  };
}
```

O Perfil e o botão "Usar outra conta" da verificação de e-mail usam o mesmo hook:

app/(tabs)/perfil.tsx

```
const handleSair = useSair();
...
<TouchableOpacity style={styles.botaoSair} onPress={handleSair}>
  <Text style={styles.textoBotaoSair}>Sair</Text>
</TouchableOpacity>
```

5.8 Ajustes visuais

Novas cores na paleta única do projeto (`styles/index.tsx`):

styles/index.tsx

```
azul: "#2563eb",          // botão "Entrar" (login com e-mail e senha)
googleBorda: "#dadce0",   // borda cinza do botão oficial do Google
googleTexto: "#1f1f1f",   // texto do botão oficial do Google
```



- O botão **Entrar** usa `cores.azul`.
- O botão do **Google** segue o padrão oficial: fundo branco, borda cinza clara, texto escuro e o logo colorido sem o círculo em volta.

6. Como testar

1. Rode `npx expo start`.
2. Na tela inicial, toque em **Não tem conta? Cadastre-se**.
3. Cadastre-se com um e-mail **real** que você consiga abrir.
4. Na tela "Confirme seu e-mail", toque em **Já confirmei**. Deve aparecer "ainda não confirmado".
5. Abra o e-mail (olhe também o spam) e clique no link.
6. Volte ao app e toque em **Já confirmei**. Você vai para o mapa.
7. Vá ao **Perfil** e toque em **Sair**. Você volta ao index, e o "voltar" não reabre o app.
8. Entre de novo com o mesmo e-mail e senha.
9. Saia, toque em **Esqueci minha senha**, crie uma nova senha pelo e-mail e entre com ela.

Para apagar contas de teste, use Authentication → Users no console do Firebase.

7. Limitações e próximos passos

- **Telas sem proteção:** a confirmação do e-mail só é checada no login. Quem ainda não confirmou consegue abrir o mapa pelo "Ver mapa sem login". Para bloquear de verdade, dá para usar o `Stack.Protected` do Expo Router e checar `emailVerified` nas regras do Firestore/backend.
- **Botão "voltar" do navegador (web):** ainda consegue reabrir `/perfil` depois do logout. A tela só mostra "Você não está logado".
- **E-mails descartáveis** (mailinator etc.) conseguem confirmar o link. Para barrá-los, seria preciso uma lista de domínios bloqueados.
- **Erros de TypeScript que já existiam** em `perfil.tsx` e `mapa.tsx`: `perfil.displayName` numa variável que pode ser texto. A correção é usar `user?.displayName` direto.